Early Evidence on Generative AI Exposure and Labor Market Outcomes

Team Members: Aesha Gandhi, Pinaki Ghosh, Shambhavi Khanna, Isaac Vergara
Stakeholder: Director of Workforce Policy, State Department of Labor
Repository: GitHub Repository

Stakeholder Summary

This memo is written for a Director of Workforce Policy at a state Department of Labor. The intended stakeholder is assumed to be comfortable interpreting applied research and labor market indicators, but not interested in econometric detail. The main priority is actionable evidence that can inform workforce planning and early-career employment policy.

Executive Summary

This memo examines whether workers in industries with higher exposure to generative AI experienced different labor market outcomes after the arrival of GenAI tools, and whether those patterns were more pronounced for younger workers. The analysis uses CPS ASEC data from 2016-2025, links those records to an industry-level AI exposure measure (`AIIE`), and compares how employment outcomes changed after the GenAI period began.

The clearest pattern is a modest worsening in unemployment outcomes after 2023 for workers in higher-AI-exposure industries. That shift appears more pronounced for workers ages 25-34 than for older workers. By contrast, no comparable wage effect is visible in the data over this short post period.

These results should be interpreted with caution. The unemployment findings are more informative than the wage findings, but neither result should be treated as definitive proof of causality. Even so, the evidence points to a practical concern for this stakeholder: labor market conditions may be softening somewhat more for younger workers in AI-exposed sectors, which makes those workers an appropriate focus for monitoring and policy attention.

Decisions To Be Made

Based on these findings, three near-term decisions appear especially relevant:

1. Whether to prioritize monitoring and support for younger workers in AI-exposed sectors, especially workers in the 25-34 range.
2. Whether to expand labor-market monitoring so that hiring, job-finding, and layoffs can be separated more clearly in future work.
3. Whether to treat wage changes as a secondary indicator and focus policy attention first on employment outcomes.

The Problem

Much of the public discussion around generative AI focuses on whether AI will eliminate jobs or lower wages. For a labor market policymaker, the more practical question is narrower: are workers in more AI-exposed industries already showing signs of weaker labor market outcomes, and are those patterns more pronounced for younger workers?

That framing matters because the analysis here covers workers ages 25-34, not only first-time labor-market entrants. The relevant policy concern is therefore broader than a “first job versus layoff” distinction. The evidence is better suited to identifying whether employment outcomes are softening more for earlier-career workers in AI-exposed industries than for older workers.

Analytical Approach

To study this question, CPS ASEC data from 2016-2025 were merged with an industry-level AI exposure measure, `AIIE`. The analysis then compares how unemployment and wage outcomes changed after the GenAI period began in industries with higher AI exposure versus industries with lower AI exposure.

The memo focuses on two outcomes:

- Unemployment
- Wage income

The analysis also looks separately at younger workers ages 25-34 to assess whether labor market changes appear more concentrated earlier in workers' careers.

Examples of relatively high-AI-exposure industries in this sample include legal services, accounting and payroll services, computer systems design, and management consulting. Lower-exposure examples include landscaping, services to buildings and dwellings, warehousing and storage, and truck transportation.

More emphasis is placed on the unemployment results because they are more consistent with the assumptions needed for this type of comparison. The wage results are still useful descriptively, but they are less persuasive as evidence of a distinct AI-related shift.

Descriptive Patterns

The descriptive trends help establish the baseline pattern before turning to the main comparisons. The plots below show annual averages with smooth pre- and post-period trend lines and uncertainty bands. High-AI-exposure industries generally have lower unemployment than low-exposure industries throughout the sample, but that gap narrows after the GenAI period begins.

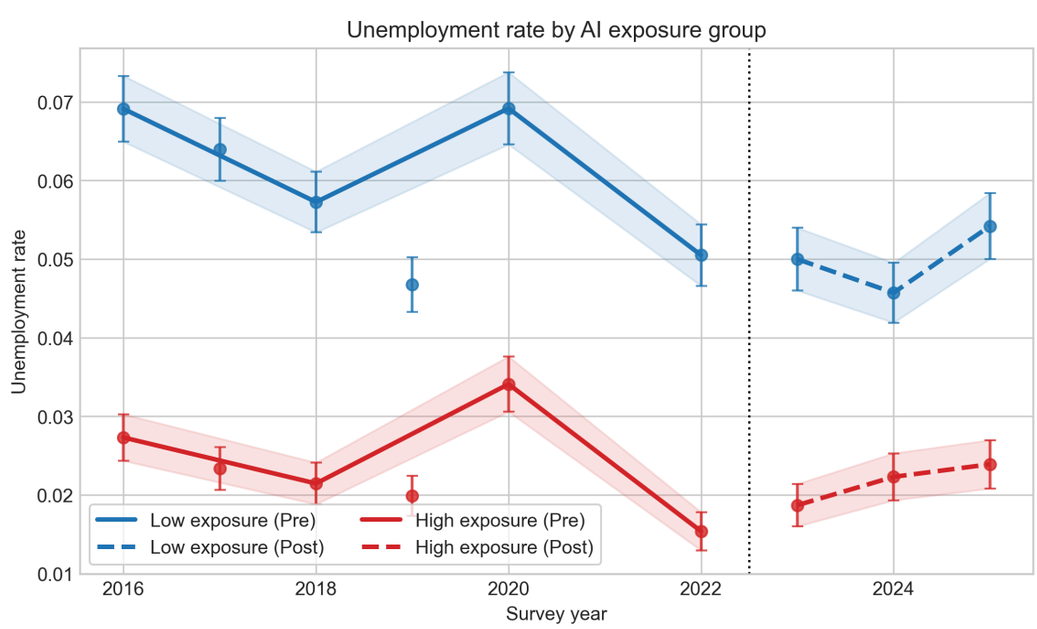


Figure 1. Annual unemployment averages by AI exposure group, with separate pre/post LOWESS fits and 95% confidence bands.

The age-focused descriptive plots point in the same direction. The relative post-period unemployment shift appears more pronounced for workers ages 25-34 than for workers ages 50-64, although the descriptive evidence alone does not identify the reason for that difference.

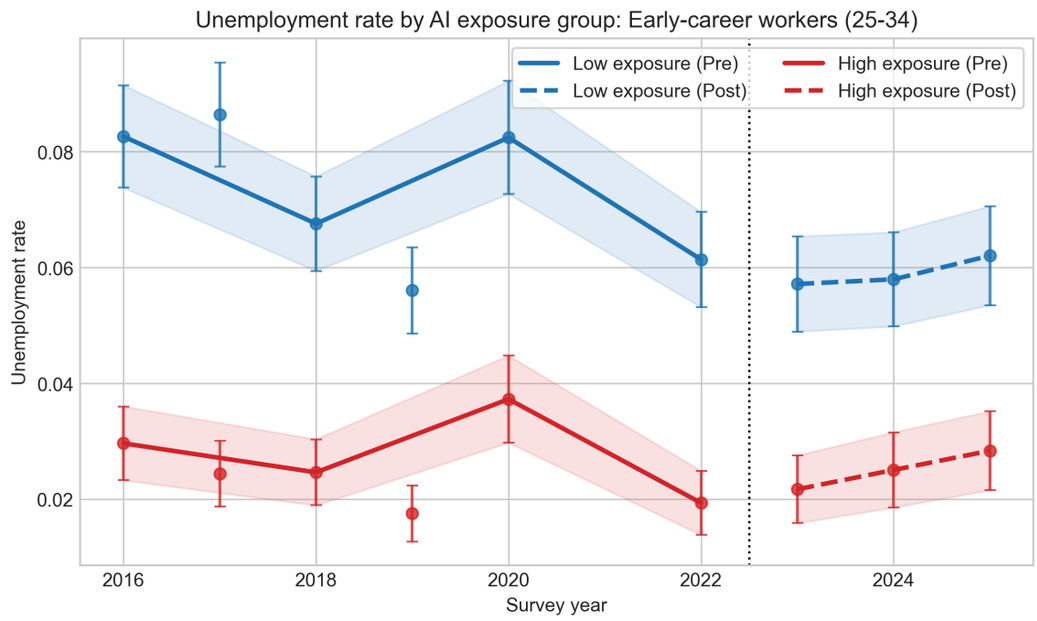


Figure 2. Annual unemployment averages for workers ages 25-34, with separate pre/post LOWESS fits and 95% confidence bands.

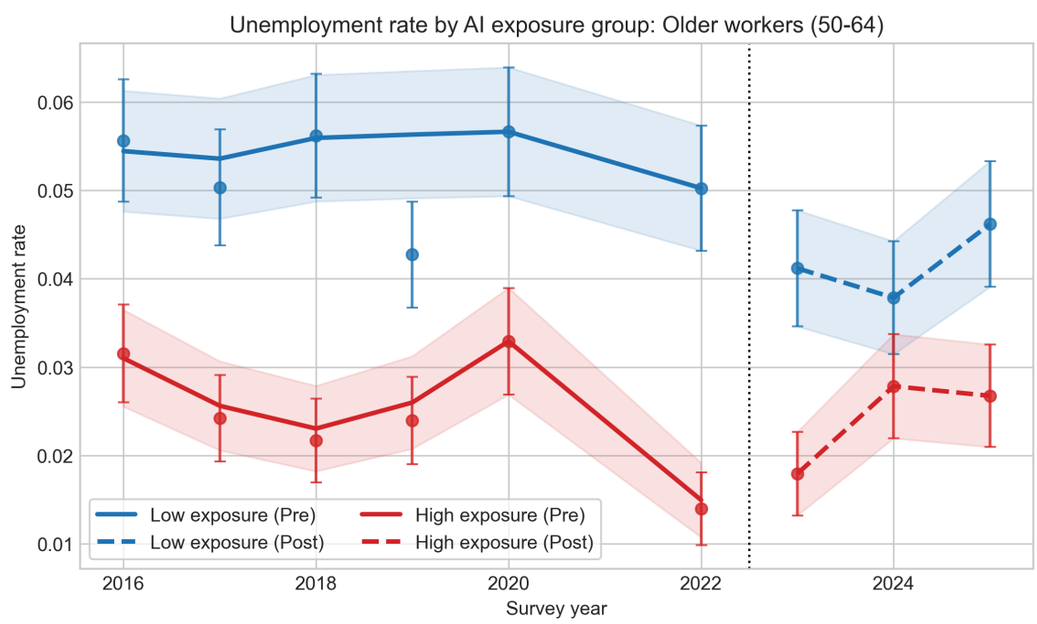


Figure 3. Annual unemployment averages for workers ages 50-64, with separate pre/post LOWESS fits and 95% confidence bands.

The wage plot is included for completeness, but it shows much less visual separation by exposure group after the GenAI period begins.

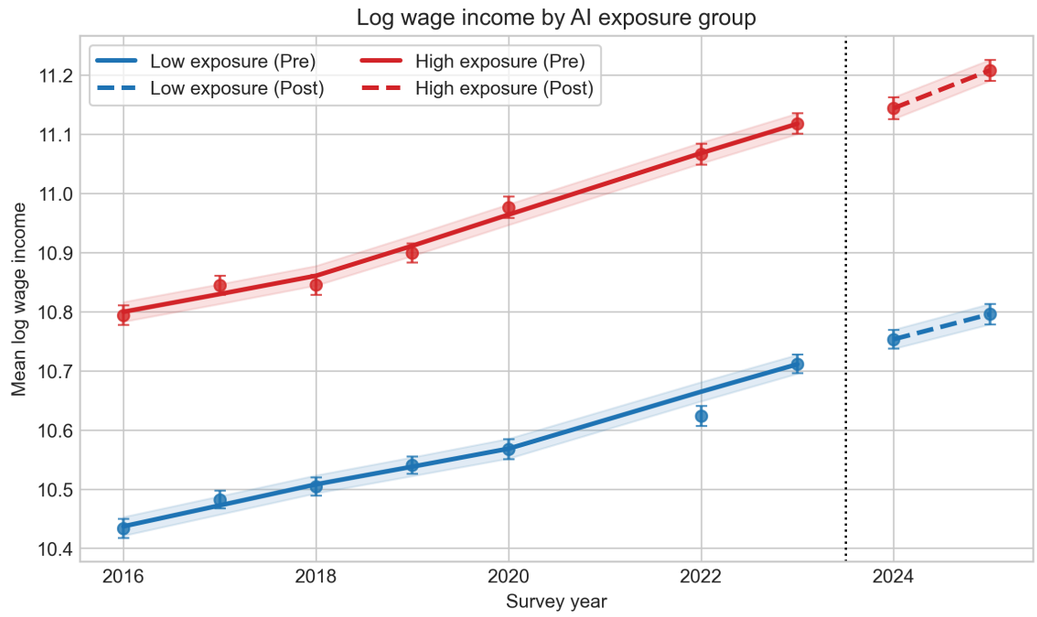


Figure 4. Annual log wage averages by AI exposure group, with separate pre/post LOWESS fits and 95% confidence bands.

Main Findings

1. Higher AI exposure is associated with a relative increase in unemployment after 2023

The main unemployment comparison suggests that unemployment outcomes worsened slightly more after 2023 in higher-AI-exposure industries than in lower-exposure industries.

This is a modest effect, not a dramatic one. It also does not mean exposed industries became high-unemployment industries overall. In the raw data, those industries still tend to have lower unemployment than less-exposed industries. The key point is that their advantage narrowed in the post period.

2. The unemployment shift is larger for workers ages 25-34

The age comparison suggests that the post-period unemployment shift is larger for workers ages 25-34 than for workers ages 35-64.

That pattern matters because it suggests the labor market impact is not evenly spread across age groups. At the same time, the analysis does not identify the exact mechanism. The safest conclusion is that younger workers in higher-AI-exposure industries experienced relatively weaker unemployment outcomes after 2023.

3. The youngest subgroup shows the largest point estimate, but the age split is not conclusive

When the younger group is split further, the estimated effect is larger for ages 25-29 than for ages 30-34. However, that difference is not statistically precise enough to support a strong claim that the youngest subgroup is distinctly more affected.

The age-split evidence is therefore best treated as suggestive. It points toward somewhat greater vulnerability among the youngest workers in the sample, but it does not establish a sharp divide within the 25-34 group.

4. No meaningful wage effect is detected

The wage analysis does not show evidence that wages changed differently after 2024 in higher-AI-exposure industries.

This should not be read as proof that AI has no wage effects. The post period is still short, and wage adjustments may take longer to appear. The wage analysis is also the weaker comparison in this memo because its pre-period patterns are less stable.

Why the Unemployment Result Is More Informative Than the Wage Result

The memo includes a diagnostic check to see whether high- and low-exposure industries were already moving differently before the GenAI period began. That check is more reassuring for unemployment than for wages.

For unemployment, the pre-period differences are smaller and less systematic, although they are not absent. For wages, the pre-period differences are more persistent, which makes it harder to interpret the later wage comparison as evidence of a distinct post-GenAI shift.

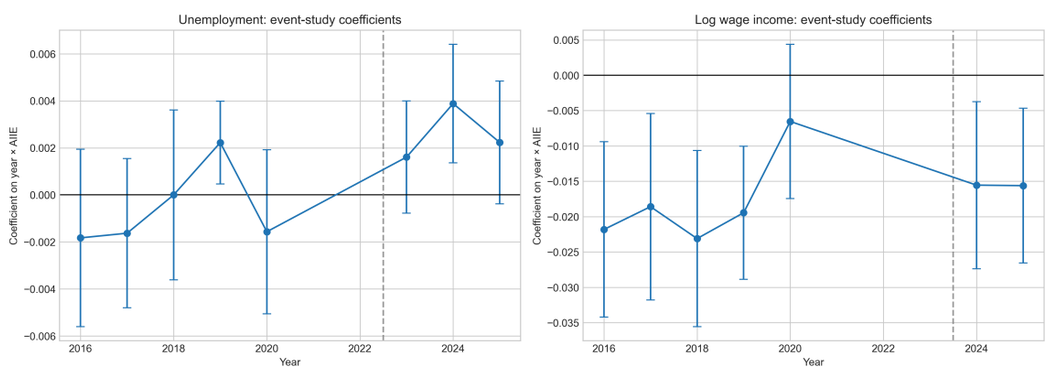


Figure 5. Year-by-year comparison of how outcomes vary with AI exposure, using 2022 as the reference year.

The robustness check that drops 2020 reaches a similar conclusion. The unemployment pattern remains visible, which reduces concern that the result is being driven entirely by pandemic-era volatility.

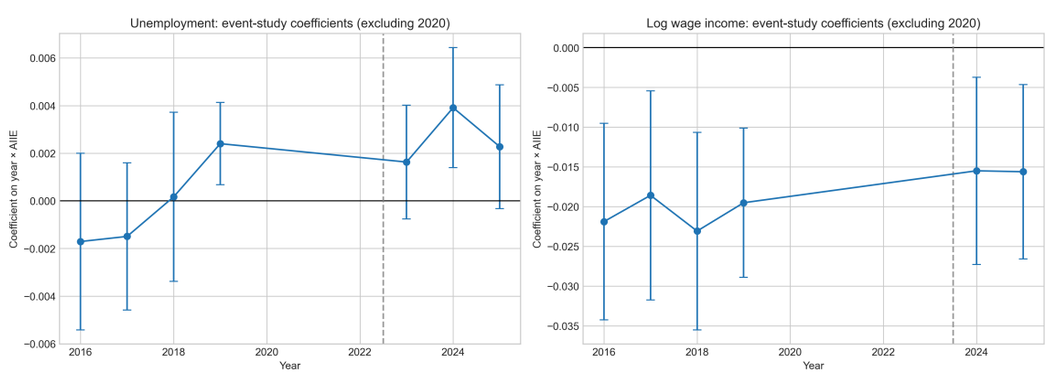


Figure 6. The same year-by-year comparison excluding 2020. The broad pattern remains similar.

Robustness

The younger-worker unemployment pattern remains visible after dropping 2020, the most unusual COVID year. That reduces concern that the main result is being driven entirely by pandemic volatility.

At the same time, the broader caution remains the same: these checks improve confidence that the pattern is not a one-year anomaly, but they do not turn the analysis

into definitive causal proof.

Limitations

This analysis has a few important limitations.

First, the CPS is a repeated cross-section, not a panel, so the same worker cannot be followed over time. The results therefore describe changes in group-level outcomes rather than individual trajectories.

Second, `AIIE` measures potential AI exposure based on industry task content, not actual AI adoption by a specific employer.

Third, the analysis cannot cleanly distinguish between reduced hiring, weaker job-finding, and increased layoffs. It shows differential employment outcomes, but not the exact reason for them.

Fourth, the post period is still short, especially for wages.

Finally, the supporting diagnostic checks are more favorable for unemployment than for wages, but they are not strong enough to remove all concern about pre-existing differences across industries.

Conclusion

The main takeaway is that labor market conditions appear to have weakened somewhat more after 2023 for younger workers in higher-AI-exposure industries. The unemployment comparison points in that direction, and the age breakdown suggests the pattern is stronger for workers ages 25-34 than for older workers.

That said, the memo should not be read as definitive proof that generative AI caused these changes. The evidence is better viewed as an early warning signal. For this stakeholder, the most practical implication is that younger workers in AI-exposed sectors warrant closer monitoring, while wage effects are not yet clearly visible in the data.

Technical Appendix

This appendix shows how the dataset was assembled and how the causal models were estimated:

- `data_merging.ipynb` for dataset construction and crosswalk logic
- `models/causal_analysis.ipynb` for sample creation, weighted descriptive analysis, parallel trends checks, DiD estimation, and robustness checks

Appendix A. Data Assembly Code

The code below shows how the CPS data were merged with industry-level AI exposure (`AIIE`) and occupation-level AI exposure (`AIOE`). Because CPS ASEC uses three different Census industry vintages across 2016–2025 (2012 codes for 2016–2019, 2017 codes for 2020–2024, 2022 codes for 2025), the assembly builds a single unified AIIE lookup that covers all three vintages via alias rows so every year resolves against the same table.

In [16]:
```
import os
from pathlib import Path
import sys
import pandas as pd
import warnings

warnings.filterwarnings("ignore")
os.environ["PYTHONWARNINGS"] = "ignore"

for candidate in [Path.cwd(), Path.cwd().parent]:
    if (candidate / "notebook_helpers.py").exists():
        if str(candidate) not in sys.path:
            sys.path.insert(0, str(candidate))
        REPO_ROOT = candidate
        break
else:
    raise FileNotFoundError("Could not find notebook_helpers.py")

from notebook_helpers import (
    ind_to_aiie,
    occ_to_aioe,
    parse_naics_patterns,
    parse_soc_pattern,
    to_canonical_ind,
)

aiie = pd.read_excel("../data/AIOE_DataAppendix.xlsx", sheet_name="Appendix
B")

crosswalk_clean = (
    pd.read_excel(
        "../data/2022-Census-Industry-Code-List-with-Crosswalk.xlsx",
        sheet_name="2022 Census Ind Code List ",
    )
    .dropna(how="all")
    .rename(
        columns={
            "Industry 2022 Description": "industry_desc",
            "2022 Census Industry Code": "census_ind_raw",
            "2022 NAICS Code": "naics_raw",
        }
    )
)

is_leaf = ~crosswalk_clean["census_ind_raw"].astype(str).str.contains("-",
na=False)
```

```
crosswalk_clean = crosswalk_clean.loc[is_leaf].copy()
crosswalk_clean["IND"] = pd.to_numeric(
    crosswalk_clean["census_ind_raw"], errors="coerce"
).astype("Int64")
crosswalk_clean = crosswalk_clean.dropna(subset=["IND"])

crosswalk_clean["naics_patterns"] = crosswalk_clean["naics_raw"].apply(
    parse_naics_patterns
)

# 2017 -> 2022: ffill both sides so split- and merge-continuation rows
aren't dropped,
# then dedupe (blank separator rows between groups otherwise leak
duplicates).
cc_17 = pd.read_excel(
    "../data/2022-Census-Industry-Code-List-with-Crosswalk.xlsx",
    sheet_name="2022 Ind Code Changes",
    header=2,
)
cc_17[["2017 Census Industry Code", "2017 Census Industry Description"]] =
cc_17[
    ["2017 Census Industry Code", "2017 Census Industry Description"]
].ffill()
cc_17[["2022 Census Industry Code", "2022 Census Industry Description"]] =
cc_17[
    ["2022 Census Industry Code", "2022 Census Industry Description"]
].ffill()
cc_17 = cc_17.dropna(
    subset=["2017 Census Industry Code", "2022 Census Industry Code"]
).astype({"2017 Census Industry Code": int, "2022 Census Industry Code":
int})
cc_17 = cc_17.drop_duplicates(
    subset=["2017 Census Industry Code", "2022 Census Industry Code"]
)

n_children_17 = cc_17.groupby("2017 Census Industry Code").size()
singletons_17 = n_children_17[n_children_17 == 1].index
splits_17 = n_children_17[n_children_17 > 1].index
ind_2017_to_2022 = dict(
    zip(
        cc_17.loc[
            cc_17["2017 Census Industry Code"].isin(singletons_17),
            "2017 Census Industry Code",
        ],
        cc_17.loc[
            cc_17["2017 Census Industry Code"].isin(singletons_17),
            "2022 Census Industry Code",
        ],
    )
)
ind_2017_splits = {
    parent: grp["2022 Census Industry Code"].tolist()
    for parent, grp in cc_17[
        cc_17["2017 Census Industry Code"].isin(splits_17)
    ].groupby("2017 Census Industry Code")
}

# 2012 -> 2017: CPS ASEC 2016-2019 data uses 2012 Census codes. This was an
error
cc_12 = pd.read_excel(
    "../data/2017-industry-code-list-with-crosswalk.xlsx",
    sheet_name="Summary of Changes",
    header=2,
).rename(columns=lambda c: c.strip())
cc_12[["2012 Census Industry Code", "2012 Description"]] = cc_12[
    ["2012 Census Industry Code", "2012 Description"]
].ffill()
cc_12[["2017 Census Industry Code", "2017 Description"]] = cc_12[
    ["2017 Census Industry Code", "2017 Description"]
].ffill()
cc_12 = cc_12.dropna(
    subset=["2012 Census Industry Code", "2017 Census Industry Code"]
).astype({"2012 Census Industry Code": int, "2017 Census Industry Code":
```

```
int})
cc_12 = cc_12.drop_duplicates(
    subset=["2012 Census Industry Code", "2017 Census Industry Code"]
)

n_children_12 = cc_12.groupby("2012 Census Industry Code").size()
singletons_12 = n_children_12[n_children_12 == 1].index
splits_12 = n_children_12[n_children_12 > 1].index
ind_2012_to_2017 = dict(
    zip(
        cc_12.loc[
            cc_12["2012 Census Industry Code"].isin(singletons_12),
            "2012 Census Industry Code",
        ],
        cc_12.loc[
            cc_12["2012 Census Industry Code"].isin(singletons_12),
            "2017 Census Industry Code",
        ],
    )
)
ind_2012_splits = {
    parent: grp["2017 Census Industry Code"].tolist()
    for parent, grp in cc_12[
        cc_12["2012 Census Industry Code"].isin(splits_12)
    ].groupby("2012 Census Industry Code")
}
```

In [17]:

```
aiie_lookup = dict(zip(aiie["NAICS"].astype(str), aiie["AIIE"]))
aiie_keys_longest_first = sorted(aiie_lookup, key=len, reverse=True)

crosswalk_clean["AIIE"] = crosswalk_clean["naics_patterns"].apply(
    lambda patterns: ind_to_aiie(patterns, aiie_lookup,
aiie_keys_longest_first)
)
ind_aiie = (
    crosswalk_clean.dropna(subset=["AIIE"])
    .astype({"IND": "int64"})[["IND", "industry_desc", "AIIE"]]
    .reset_index(drop=True)
)

# Add alias rows so 2017- and 2012-vintage CPS codes resolve against the
same table.
# 1:1 renames/merges inherit the child's AIIE; split parents get the mean
of children.
aiie_by_2022 = dict(zip(ind_aiie["IND"], ind_aiie["AIIE"]))

alias_2017 = []
for c17, c22 in ind_2017_to_2022.items():
    if c17 in aiie_by_2022:
        continue
    if c22 in aiie_by_2022:
        alias_2017.append(
            {"IND": c17, "industry_desc": "(2017 alias)", "AIIE":
aiie_by_2022[c22]}
        )
for parent, kids in ind_2017_splits.items():
    vals = [aiie_by_2022[k] for k in kids if k in aiie_by_2022]
    if vals:
        alias_2017.append(
            {
                "IND": parent,
                "industry_desc": f"(2017 parent of {len(kids)} 2022
codes)",
                "AIIE": sum(vals) / len(vals),
            }
        )

aiie_by_2017 = dict(aiie_by_2022)
for row in alias_2017:
    aiie_by_2017[row["IND"]] = row["AIIE"]

alias_2012 = []
for c12, c17 in ind_2012_to_2017.items():
```

```
    if c12 in aiie_by_2017:
        continue
    if c17 in aiie_by_2017:
        alias_2012.append(
            {"IND": c12, "industry_desc": "(2012 alias)", "AIIE":
aiie_by_2017[c17]}
        )
for parent, kids in ind_2012_splits.items():
    vals = [aiie_by_2017[k] for k in kids if k in aiie_by_2017]
    if vals:
        alias_2012.append(
            {
                "IND": parent,
                "industry_desc": f"(2012 parent of {len(kids)} 2017
codes)",
                "AIIE": sum(vals) / len(vals),
            }
        )

ind_aiie = pd.concat(
    [ind_aiie, pd.DataFrame(alias_2017), pd.DataFrame(alias_2012)],
    ignore_index=True,
)

df = pd.read_csv("../data/cps_00001.csv")

df["IND_2022"] = df["IND"].map(
    lambda x: to_canonical_ind(x, ind_2012_to_2017, ind_2017_to_2022)
)
df_aiie = df.merge(ind_aiie, on="IND", how="left")
```

In [18]:

```
aioe = pd.read_excel("../data/AIOE_DataAppendix.xlsx", sheet_name="Appendix
A")

occ_crosswalk = (
    pd.read_excel(
        "../data/2018-occupation-code-list-and-crosswalk.xlsx",
        sheet_name="2018 Census Occ Code List",
        header=0,
    )
    .dropna(how="all")
    .rename(
        columns={
            "2018 Census Title ": "occ_desc",
            "2018 Census Code": "census_occ_raw",
            "2018 SOC Code": "soc_raw",
        }
    )
)

is_leaf = ~occ_crosswalk["census_occ_raw"].astype(str).str.contains("-",
na=False)
occ_crosswalk = occ_crosswalk.loc[is_leaf].copy()
occ_crosswalk["OCC"] = pd.to_numeric(
    occ_crosswalk["census_occ_raw"], errors="coerce"
).astype("Int64")
occ_crosswalk = occ_crosswalk.dropna(subset=["OCC"])

occ_crosswalk["soc_pattern"] =
occ_crosswalk["soc_raw"].apply(parse_soc_pattern)

aioe_lookup = dict(zip(aioe["SOC Code"].astype(str), aioe["AIOE"]))
aioe_keys_longest_first = sorted(aioe_lookup, key=len, reverse=True)

occ_crosswalk["AIOE"] = occ_crosswalk["soc_pattern"].apply(
    lambda pattern: occ_to_aioe(pattern, aioe_lookup,
aioe_keys_longest_first)
)
occ_aioe = (
    occ_crosswalk.dropna(subset=["AIOE"])
    .astype({"OCC": "int64"})[["OCC", "occ_desc", "AIOE"]]
```

```
    .reset_index(drop=True)
)
```

In [19]:
```
occ_changes = (
    pd.read_excel(
        "../data/2018-occupation-code-list-and-crosswalk.xlsx",
        sheet_name="Occ Code Changes",
        header=2,
    )
    .rename(
        columns={
            "2010 Census OCC Code": "occ_2010",
            "2010 Census Title": "title_2010",
            "New 2018 Census OCC Code": "occ_2018",
        }
    )
    .dropna(how="all")
)

occ_changes[["occ_2010", "title_2010"]] = occ_changes[
    ["occ_2010", "title_2010"]
].ffill()
occ_changes = occ_changes.dropna(subset=["occ_2010", "occ_2018"]).astype(
    {"occ_2010": int, "occ_2018": int}
)

occ_2018_to_aioe = dict(zip(occ_aioe["OCC"], occ_aioe["AIOE"]))
occ_changes["AIOE"] = occ_changes["occ_2018"].map(occ_2018_to_aioe)

pre2018 = (
    occ_changes[~occ_changes["occ_2010"].isin(occ_aioe["OCC"])]
    .dropna(subset=["AIOE"])
    .groupby("occ_2010", as_index=False)
    .agg(occ_desc=("title_2010", "first"), AIOE=("AIOE", "mean"))
    .rename(columns={"occ_2010": "OCC"})
)

occ_aioe_full = pd.concat([occ_aioe, pre2018], ignore_index=True)
df_aiie = df_aiie.merge(occ_aioe_full, on="OCC", how="left")

df_aiie.to_csv("../data/data_with_aiie.csv", index=False)
```

Appendix B. Analysis Setup and Variable Construction

The next blocks reproduce the final causal analysis setup, including reproducibility settings, sample restrictions, variable definitions, and helper functions used throughout the analysis.

Analysis Frame and Key Variables

The final analysis uses CPS ASEC repeated cross-sections from 2016-2025 merged with industry-level AI exposure, `AIIE`. The analytical frame applies the following restrictions and variable definitions:

- Working-age sample: ages 25-64
- Exposure requirement: non-missing `AIIE`
- Unemployment sample: individuals in the labor force
- Wage sample: employed individuals with positive wage income
- Main wage DiD sample: same as the wage sample, excluding 2023 as a transition year
- Survey weights: `ASECWT`
- Clustered standard errors: by `IND_2022`
- `post_unemployment = 1` for `YEAR >= 2023`
- `post_wage = 1` for `YEAR >= 2024`
- `early_career = 1` for ages 25-34
- `log_wage = log(INCWAGE)` for positive wage observations

For plots only, `AIIE` is also split into low- and high-exposure groups using the 25th and 75th percentiles. The regressions use continuous `AIIE`, not the grouped version.

In [20]:
```
from pathlib import Path
import random
import sys

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

for candidate in [Path.cwd(), Path.cwd().parent]:
    if (candidate / "notebook_helpers.py").exists():
        if str(candidate) not in sys.path:
            sys.path.insert(0, str(candidate))
        REPO_ROOT = candidate
        break
else:
    raise FileNotFoundError("Could not find notebook_helpers.py")

from notebook_helpers import (
    coef_table,
    extract_event_study_table,
    fit_event_study,
    joint_zero_test,
    plot_event_study,
    plot_event_study_on_axis,
    plot_trends,
    run_wls,
    summarize_sample,
    wald_difference_test,
    weighted_mean,
    weighted_mean_se,
    yearly_group_stats,
)

SEED = 42
random.seed(SEED)
np.random.seed(SEED)

plt.style.use("seaborn-v0_8-whitegrid")
pd.set_option("display.max_columns", 100)
pd.set_option("display.float_format", lambda x: f"{x:,.4f}")

DATA_PATH = Path("../data/data_with_aiie.csv")
data = pd.read_csv(DATA_PATH)
```

In [21]:
```
df = data.copy()

df = df[df["AGE"].between(25, 64)].copy()
df = df[df["AIIE"].notna()].copy()

df["employed"] = df["EMPSTAT"].isin([10, 12]).astype(int)
df["unemployed"] = df["EMPSTAT"].isin([20, 21, 22]).astype(int)
df["in_labor_force"] = (df["LABFORCE"] == 2).astype(int)

df["post_unemployment"] = (df["YEAR"] >= 2023).astype(int)
df["post_wage"] = (df["YEAR"] >= 2024).astype(int)

df["early_career"] = df["AGE"].between(25, 34).astype(int)
df["older_worker"] = df["AGE"].between(50, 64).astype(int)

df["positive_wage"] = (df["INCWAGE"] > 0).astype(int)
df["log_wage"] = np.where(df["INCWAGE"] > 0, np.log(df["INCWAGE"]), np.nan)

q1 = df["AIIE"].quantile(0.25)
q3 = df["AIIE"].quantile(0.75)
df["exposure_group"] = np.select(
    [df["AIIE"] <= q1, df["AIIE"] >= q3],
    ["Low exposure", "High exposure"],
    default="Middle exposure",
```

```
)

unemp_sample = df[df["in_labor_force"] == 1].copy()
wage_sample = df[(df["employed"] == 1) & (df["positive_wage"] == 1)].copy()
wage_did_sample = wage_sample[wage_sample["YEAR"] != 2023].copy()
```

In [22]:
```
from notebook_helpers import (
    weighted_mean,
    weighted_mean_se,
    summarize_sample,
    yearly_group_stats,
    smooth_segment,
    plot_trends,
    run_wls,
    coef_table,
    wald_difference_test,
)
```

Appendix C. Core Estimation Code

This section includes the code used for the descriptive plots, event-study diagnostics, main difference-in-differences regressions, and the age-heterogeneity specifications.

Difference-in-Differences Specifications

The baseline unemployment difference-in-differences model is:

$$\begin{aligned} unemployed_{it} = \beta_0 + \beta_1 AIIE_j + \beta_2 post_t^{unemp} \\ + \beta_3 \left(AIIE_j \times post_t^{unemp}\right) + \beta_4 AGE_i \\ + \sum_e \delta_e 1\{EDUC_i = e\} + \varepsilon_{it} \end{aligned}$$

The unemployment heterogeneity model adds a triple interaction:

$$\begin{aligned} unemployed_{it} = \alpha_0 + \alpha_1 AIIE_j + \alpha_2 post_t^{unemp} + \alpha_3 early_career_i \\ + \alpha_4 \left(AIIE_j \times post_t^{unemp}\right) \\ + \alpha_5 \left(AIIE_j \times early_career_i\right) \\ + \alpha_6 \left(post_t^{unemp} \times early_career_i\right) \\ + \alpha_7 \left(AIIE_j \times post_t^{unemp} \times early_career_i\right) \\ + \alpha_8 AGE_i + \sum_e \rho_e 1\{EDUC_i = e\} + u_{it} \end{aligned}$$

The baseline wage difference-in-differences model is:

$$\begin{aligned} \log(wage_{it}) = \gamma_0 + \gamma_1 AIIE_j + \gamma_2 post_t^{wage} \\ + \gamma_3 \left(AIIE_j \times post_t^{wage}\right) + \gamma_4 AGE_i \\ + \sum_e \kappa_e 1\{EDUC_i = e\} + \nu_{it} \end{aligned}$$

The wage heterogeneity model parallels the unemployment triple-interaction setup:

$$\begin{aligned} \log(wage_{it}) = \eta_0 + \eta_1 AIIE_j + \eta_2 post_t^{wage} + \eta_3 early_career_i \\ + \eta_4 \left(AIIE_j \times post_t^{wage}\right) \\ + \eta_5 \left(AIIE_j \times early_career_i\right) \\ + \eta_6 \left(post_t^{wage} \times early_career_i\right) \\ + \eta_7 \left(AIIE_j \times post_t^{wage} \times early_career_i\right) \\ + \eta_8 AGE_i + \sum_e \phi_e 1\{EDUC_i = e\} + \omega_{it} \end{aligned}$$

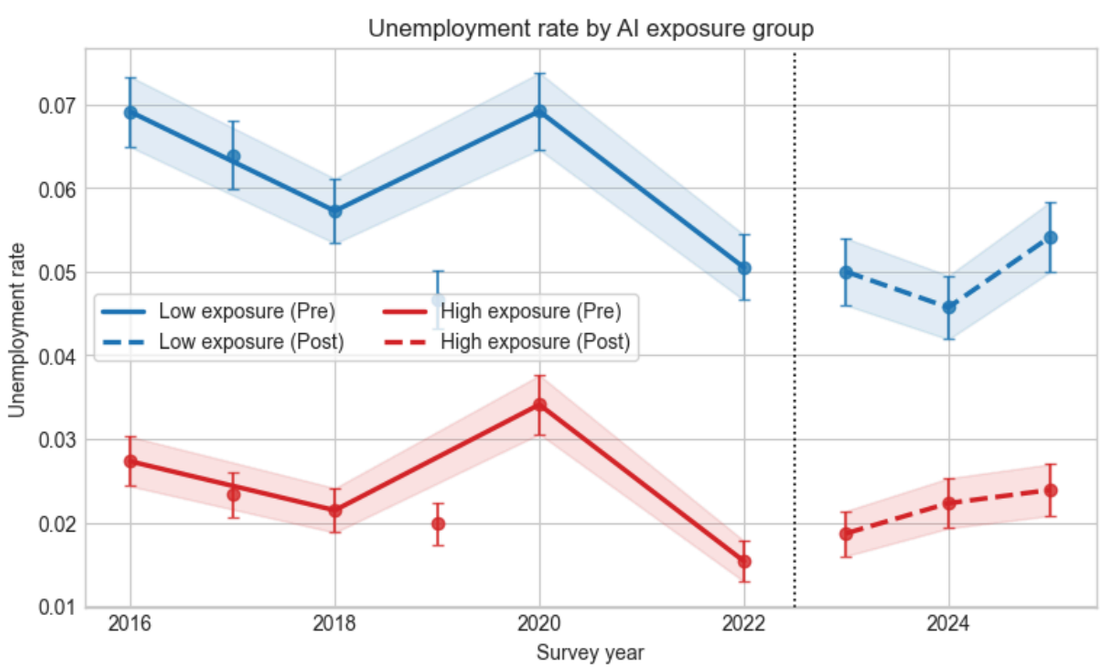
Variable definitions:

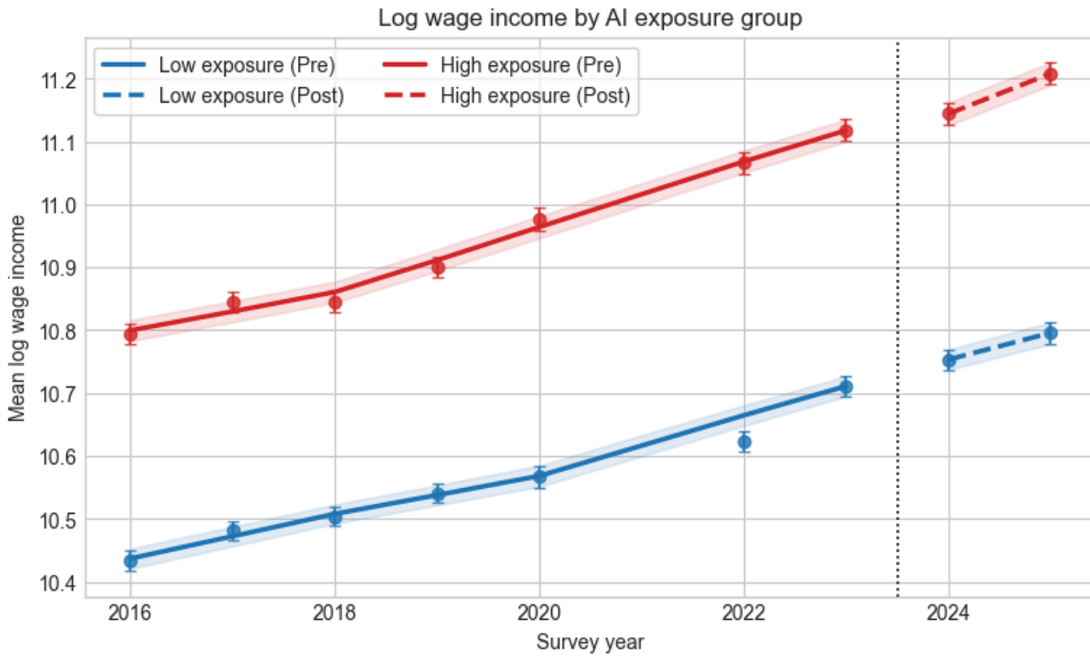
- $unemployed_{it}$: indicator equal to 1 if individual i is unemployed in year t
- $\log(wage_{it})$: log wage income for individual i in year t

- $AIIE_j$: AI Industry Exposure for industry j
- $post_t^{unemp}$: indicator equal to 1 in 2023-2025
- $post_t^{wage}$: indicator equal to 1 in 2024-2025
- $early_career_i$: indicator equal to 1 for ages 25-34
- AGE_i: age in years
- $1\{EDUC_i = e\}$: education-category controls

All models are estimated using CPS ASEC weights and cluster-robust standard errors at the 2022 Census industry level.

In [23]:
```
coverage_table = pd.DataFrame(
    {
        "sample": [
            "Working-age analysis frame",
            "Unemployment sample",
            "Wage sample",
            "Main wage DiD sample",
        ],
        "n_rows": [len(df), len(unemp_sample), len(wage_sample),
len(wage_did_sample)],
    }
)
coverage_table

fig_unemp, ax_unemp, unemp_plot_df = plot_trends(
    unemp_sample,
    value_col="unemployed",
    title="Unemployment rate by AI exposure group",
    ylabel="Unemployment rate",
    post_start=2023,
)
plt.show()

fig_wage, ax_wage, wage_plot_df = plot_trends(
    wage_sample,
    value_col="log_wage",
    title="Log wage income by AI exposure group",
    ylabel="Mean log wage income",
    post_start=2024,
)
plt.show()
```





Now, we will dive into a check for parallel trends.

Why Check Parallel Trends?

The descriptive figures are useful for showing the broad pattern in the data, but they are not enough on their own to justify a difference-in-differences interpretation. The key identifying assumption behind the DiD design is that, absent the post-2022 expansion of generative AI, outcomes in higher- and lower-exposure industries would have continued to move along similar underlying paths.

This is why we now dive into an event-study version of the model. The event-study check asks whether the relationship between `AIIE` and the outcome was already shifting before the post period began. If those pre-period differences are small, that makes the DiD setup more credible. If they are large or systematic, then the post-period DiD estimates are harder to interpret causally because some of the gap may reflect pre-existing divergence rather than a post-2022 change.

Parallel Trends and Event-Study Diagnostic

Parallel trends are evaluated with an event-study specification that replaces the single post-period interaction with year-specific interactions between `AIIE` and survey year:

$$
\begin{aligned}
Y_{it} = \theta_0 &+ \sum_{\tau \neq 2022} \theta_\tau \left(AIIE_j \times 1\{YEAR_t = \tau\} \right) \\
&+ \gamma_j + \sum_{\tau \neq 2022} \lambda_\tau 1\{YEAR_t = \tau\} \\
&+ \sum_k \psi_k X_{kit} + \varepsilon_{it}
\end{aligned}
$$

where Y_{it} is either unemployment or log wage income, and X_{kit} includes age and education controls. This is not a staggered-adoption design. The omitted year is 2022, so each coefficient shows how the relationship between `AIIE` and the outcome differs relative to 2022.

Interpretation of the diagnostic:

- If pre-period coefficients are close to zero, that supports the parallel-trends assumption.
- If pre-period coefficients are systematically different from zero, that suggests pre-existing differential trends.

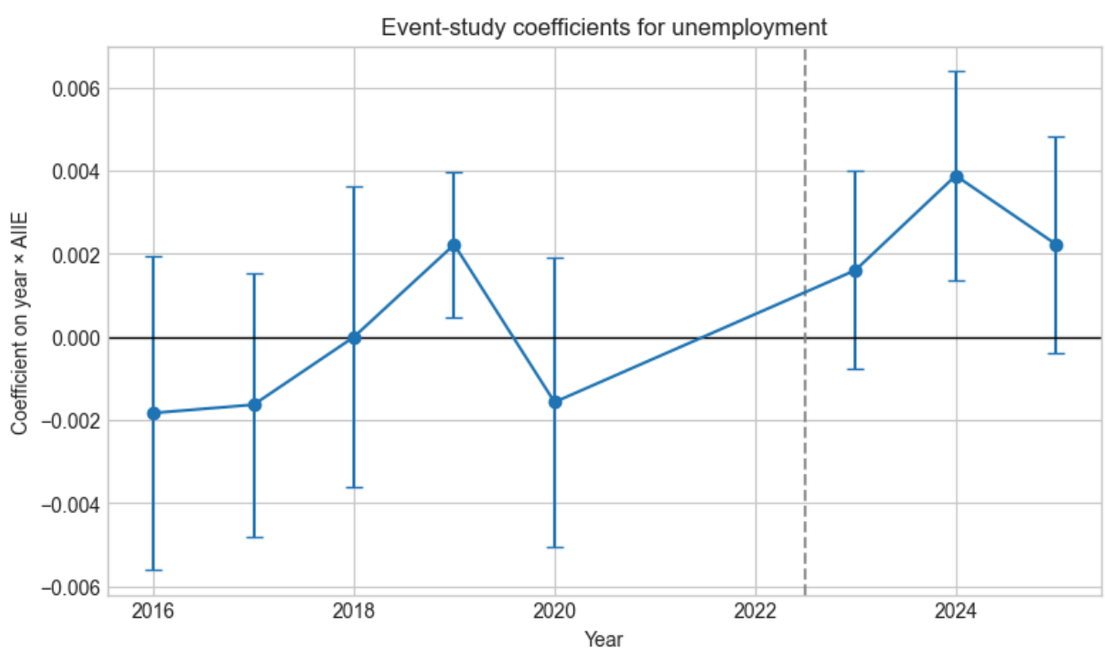
- The joint pre-trend test evaluates whether all pre-period `year × AIIE` terms are zero together.

Neither outcome fully passes this diagnostic. The unemployment specification still looks more informative than the wage specification because its pre-period deviations are smaller and less systematic, but it should still be treated as suggestive rather than definitive.

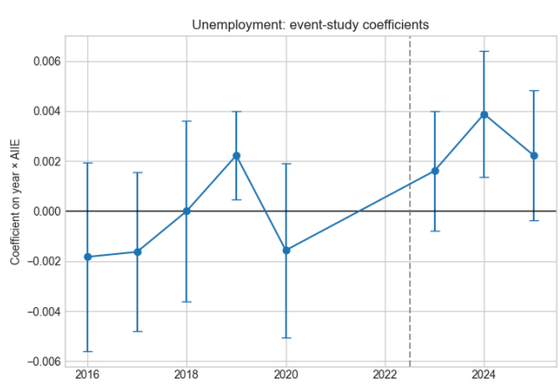
In [24]:
```
from notebook_helpers import (
    fit_event_study,
    extract_event_study_table,
    joint_zero_test,
    plot_event_study_on_axis,
    plot_event_study,
)
```

In [25]:
```
unemp_event = fit_event_study(unemp_sample, "unemployed",
reference_year=2022)
unemp_event_table = extract_event_study_table(unemp_event,
reference_year=2022)
display(unemp_event_table)

fig_unemp_event, ax_unemp_event = plot_event_study(
    unemp_event_table,
    title="Event-study coefficients for unemployment",
    post_start=2023,
)
plt.show()

unemp_pre_terms = unemp_event_table.loc[
    unemp_event_table["year"] < 2023, "term"
].tolist()
joint_zero_test(unemp_event, unemp_pre_terms)

wage_event = fit_event_study(wage_did_sample, "log_wage",
reference_year=2022)
wage_event_table = extract_event_study_table(wage_event,
reference_year=2022)
display(wage_event_table)

fig_wage_event, ax_wage_event = plot_event_study(
    wage_event_table,
    title="Event-study coefficients for log wage income",
    post_start=2024,
)
plt.show()

wage_pre_terms = wage_event_table.loc[
    wage_event_table["year"] < 2024, "term"
].tolist()
joint_zero_test(wage_event, wage_pre_terms)

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))
plot_event_study_on_axis(
    unemp_event_table,
    ax1,
    "Unemployment: event-study coefficients",
    post_start=2023,
)
plot_event_study_on_axis(
    wage_event_table,
    ax2,
    "Log wage income: event-study coefficients",
    post_start=2024,
)
plt.tight_layout()
plt.show()
```

	term	year	coef	std_err	ci_low	ci_high	p_value
0	C(YEAR, Treatment(reference=2022))[T.2016]:AIIE	2016	-0.0018	0.0019	-0.0056	0.0019	0.3423
1	C(YEAR, Treatment(reference=2022))[T.2017]:AIIE	2017	-0.0016	0.0016	-0.0048	0.0015	0.3144
2	C(YEAR, Treatment(reference=2022))[T.2018]:AIIE	2018	0.0000	0.0018	-0.0036	0.0036	0.9996
3	C(YEAR, Treatment(reference=2022))[T.2019]:AIIE	2019	0.0022	0.0009	0.0005	0.0040	0.0131
4	C(YEAR, Treatment(reference=2022))[T.2020]:AIIE	2020	-0.0016	0.0018	-0.0051	0.0019	0.3792
5	C(YEAR, Treatment(reference=2022))[T.2023]:AIIE	2023	0.0016	0.0012	-0.0008	0.0040	0.1868
6	C(YEAR, Treatment(reference=2022))[T.2024]:AIIE	2024	0.0039	0.0013	0.0014	0.0064	0.0025
7	C(YEAR, Treatment(reference=2022))[T.2025]:AIIE	2025	0.0022	0.0013	-0.0004	0.0048	0.0934



	term	year	coef	std_err	ci_low	ci_high	p_value
0	C(YEAR, Treatment(reference=2022))[T.2016]:AIIE	2016	-0.0218	0.0063	-0.0342	-0.0094	0.0006
1	C(YEAR, Treatment(reference=2022))[T.2017]:AIIE	2017	-0.0186	0.0067	-0.0318	-0.0054	0.0057
2	C(YEAR, Treatment(reference=2022))[T.2018]:AIIE	2018	-0.0231	0.0063	-0.0355	-0.0107	0.0003
3	C(YEAR, Treatment(reference=2022))[T.2019]:AIIE	2019	-0.0194	0.0048	-0.0289	-0.0100	0.0001
4	C(YEAR, Treatment(reference=2022))[T.2020]:AIIE	2020	-0.0065	0.0056	-0.0174	0.0044	0.2401
5	C(YEAR, Treatment(reference=2022))[T.2024]:AIIE	2024	-0.0155	0.0060	-0.0274	-0.0037	0.0099
6	C(YEAR, Treatment(reference=2022))[T.2025]:AIIE	2025	-0.0156	0.0056	-0.0266	-0.0047	0.0052



In [26]:
```
m_unemp_base = run_wls(
    "unemployed ~ AIIE * post_unemployment + AGE + C(EDUC)",
    unemp_sample,
)
coef_table(m_unemp_base, keep="AIIE|post")

m_unemp_triple = run_wls(
    "unemployed ~ AIIE * post_unemployment * early_career + AGE + C(EDUC)",
    unemp_sample,
)
```

```
coef_table(m_unemp_triple, keep="AIIE|post|early")

m_wage_base = run_wls(
    "log_wage ~ AIIE * post_wage + AGE + C(EDUC)",
    wage_did_sample,
)
coef_table(m_wage_base, keep="AIIE|post")

m_wage_triple = run_wls(
    "log_wage ~ AIIE * post_wage * early_career + AGE + C(EDUC)",
    wage_did_sample,
)
coef_table(m_wage_triple, keep="AIIE|post|early")
```

Out[26]:

	coef	std_err	p_value
AIIE	0.0190	0.0469	0.6853
post_wage	0.2253	0.0075	0.0000
AIIE:post_wage	-0.0010	0.0081	0.8975
early_career	-0.1838	0.0126	0.0000
AIIE:early_career	-0.0087	0.0193	0.6512
post_wage:early_career	0.0484	0.0096	0.0000
AIIE:post_wage:early_career	0.0123	0.0077	0.1092

Appendix D. Robustness and Heterogeneity Code

The final blocks show the robustness exercise that excludes 2020 and the age-split specification that isolates whether the unemployment effect is driven more by workers ages 25-29 than by workers ages 30-34.

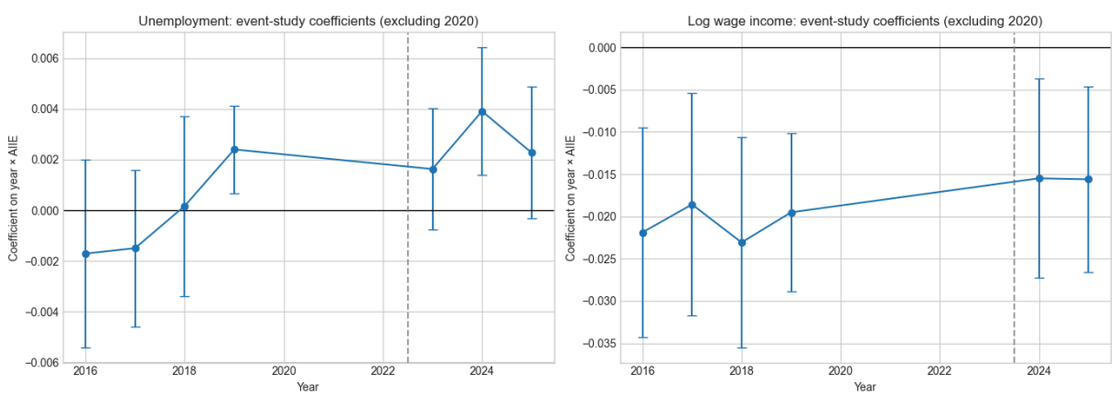
In [27]:

```
unemp_no20 = unemp_sample[unemp_sample["YEAR"] != 2020].copy()
wage_no20 = wage_did_sample[wage_did_sample["YEAR"] != 2020].copy()

m_unemp_no20 = run_wls(
    "unemployed ~ AIIE * post_unemployment * early_career + AGE + C(EDUC)",
    unemp_no20,
)
m_wage_no20 = run_wls(
    "log_wage ~ AIIE * post_wage * early_career + AGE + C(EDUC)",
    wage_no20,
)

display(coef_table(m_unemp_no20, keep="AIIE|post|early"))
display(coef_table(m_wage_no20, keep="AIIE|post|early"))

unemp_no20_event = fit_event_study(unemp_no20, "unemployed",
reference_year=2022)
unemp_no20_event_table = extract_event_study_table(unemp_no20_event,
reference_year=2022)
wage_no20_event = fit_event_study(wage_no20, "log_wage",
reference_year=2022)
wage_no20_event_table = extract_event_study_table(wage_no20_event,
reference_year=2022)

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))
plot_event_study_on_axis(
    unemp_no20_event_table,
    ax1,
    "Unemployment: event-study coefficients (excluding 2020)",
    post_start=2023,
)
plot_event_study_on_axis(
    wage_no20_event_table,
    ax2,
    "Log wage income: event-study coefficients (excluding 2020)",
    post_start=2024,
```

```
)
plt.tight_layout()
plt.show()
```

	coef	std_err	p_value
AIIE	-0.0058	0.0019	0.0019
post_unemployment	-0.0022	0.0016	0.1646
AIIE:post_unemployment	0.0016	0.0013	0.2140
early_career	0.0087	0.0015	0.0000
AIIE:early_career	-0.0069	0.0015	0.0000
post_unemployment:early_career	-0.0017	0.0017	0.2919
AIIE:post_unemployment:early_career	0.0037	0.0014	0.0078

	coef	std_err	p_value
AIIE	0.0167	0.0465	0.7194
post_wage	0.2346	0.0077	0.0000
AIIE:post_wage	0.0012	0.0085	0.8927
early_career	-0.1817	0.0137	0.0000
AIIE:early_career	-0.0085	0.0194	0.6615
post_wage:early_career	0.0469	0.0100	0.0000
AIIE:post_wage:early_career	0.0120	0.0083	0.1488



In [28]:
```
unemp_sample["age_group"] = pd.cut(
    unemp_sample["AGE"],
    bins=[24, 29, 34, 64],
    labels=["25-29", "30-34", "35-64"],
)

m_age_split = run_wls(
    "unemployed ~ AIIE * post_unemployment * C(age_group,
Treatment(reference='35-64'))"
    " + AGE + C(EDUC)",
    unemp_sample,
)

key_terms = [
    k
    for k in m_age_split.params.index
    if "AIIE:post_unemployment:C(age_group" in k
]
age_split_table = coef_table(m_age_split).loc[key_terms].copy()

term_25_29 = next(k for k in key_terms if "[T.25-29]" in k)
term_30_34 = next(k for k in key_terms if "[T.30-34]" in k)
age_gap_test = wald_difference_test(
    m_age_split,
    term_25_29,
    term_30_34,
    "25-29 coefficient minus 30-34 coefficient",
```

```
)

display(age_split_table)
display(age_gap_test)
```

	coef	std_err	p_value
AIIE:post_unemployment:C(age_group, Treatment(reference='35-64'))[T.25-29]	0.0045	0.0020	0.0274
AIIE:post_unemployment:C(age_group, Treatment(reference='35-64'))[T.30-34]	0.0024	0.0017	0.1712

	comparison	difference	wald_stat	p_value
0	25-29 coefficient minus 30-34 coefficient	0.0022	0.6477	0.4210

Appendix E. Submission Export

The cell below uses `nbconvert` to export the notebook to HTML for submission.

In [30]:
```
from pathlib import Path
import subprocess
import sys

notebook = Path("data_science_memo.ipynb")
output_name = "data_science_memo_submission"

cmd = [
    sys.executable,
    "-m",
    "jupyter",
    "nbconvert",
    "--to",
    "html",
    "--output",
    output_name,
    "--log-level=ERROR",
    str(notebook),
]

print("Running:", " ".join(cmd))
subprocess.run(
    cmd, check=True, stdout=subprocess.PIPE, stderr=subprocess.PIPE,
text=True
)
print(f"Created {output_name}.html")
```

```
Running: /Users/aeshagandhi/Downloads/MIDS-Sp26/UDS/UDS-
Project/.venv/bin/python -m jupyter nbconvert --to html --output
data_science_memo_submission --log-level=ERROR data_science_memo.ipynb
Created data_science_memo_submission.html
```

In [ ]: